Delivery Partner Dispatch System Design

Delivery Partner Assignment at a Quick Commerce Dark Store

A system design framework for order-to-rider dispatch and supply-shortage handling

---

Table of Contents

1. Reframing the Problem
2. Data You Need
3. Questions to Ask If This Data Doesn't Exist Yet
4. System Architecture: The Dispatch Pipeline
5. The Matching Algorithm
6. When No Rider Is Available: The Escalation Ladder
7. Factors You Didn't Mention — But That Matter
8. Constraints That Shape This System
9. Base Assumptions

---

1. Reframing the Problem

This is a **real-time task-assignment / bipartite matching problem**, nested inside a **queueing and supply-shortage problem** for the edge case. Two distinct sub-problems:

1. **The happy path** — one or more riders idle at the store, one or more orders ready to go: which order goes to which rider, and should any be bundled into one trip?
2. **The shortage path** — demand for dispatch exceeds available supply at that instant: what does the system do while it's short a rider?

Both need to be solved together because the second is really just "what the matching engine does when its input set of idle riders is empty" — it's not a separate system, it's a **degraded-mode behavior** of the same one.

---

2. Data You Need

Category	Specifics
Rider real-time state	Idle / en-route-to-store / delivering / on-break, live GPS location, geofence check-in/out events at the store
Rider capacity	Vehicle type (bike/scooter/bicycle/on-foot), max concurrent orders (bag size/volume), current load if already carrying an order
Rider shift & compliance data	Shift start/end time, hours worked so far, mandatory break due, acceptance/rejection history
Order data	Ready timestamp, promised SLA/delivery deadline, item weight/volume, cold-chain/perishability flag, customer address & zone, customer tier
Historical assignment data	Offer-to-accept time, accept/reject/timeout rates per rider, pickup-to-delivery time, no-show rate
Zone-level demand & supply history	Order density and rider availability by time-of-day and zone — needed for forecasting and for deciding when to trigger incentives
Neighboring store visibility	Rider pool and queue state of nearby dark stores with overlapping delivery catchments (only relevant if dispatch is orchestrated centrally, not per-store)
Incentive/surge data	Historical payout amounts and their effect on rider login/response times

Weather data	Rain measurably reduces rider supply (fewer riders willing to ride) while simultaneously increasing order demand — a compounding factor worth tracking explicitly
Third-party fleet aggregator data	If used for overflow — their API availability, per-order pricing, and SLA reliability

---

3. Questions to Ask If This Data Doesn't Exist Yet

- **"Do we track real-time rider state with geofencing at the store, or is 'available' self-reported by the rider?"** Self-reported availability is unreliable — riders can mark themselves idle while actually elsewhere. Geofence-verified check-in should be the source of truth if it isn't already.
- **"Do we log the full offer→accept/reject/timeout cascade per rider, or just the final assignment?"** Without the full cascade, you can't measure rider acceptance behavior or tune offer-timeout windows — this data is essential for improving match quality over time.
- **"Is dispatch decided independently per store, or orchestrated centrally across stores with overlapping catchments?"** This single answer determines whether cross-store rider borrowing (Section 6) is even possible today or needs new infrastructure.
- **"Do we have a rider incentive/surge engine, and can it be triggered automatically based on real-time shortage signals?"** If incentives are only set manually by ops teams, there's a meaningful latency gap between a shortage occurring and any corrective action.
- **"Do we have a fallback relationship with a third-party fleet aggregator for overflow capacity?"** If not, worth evaluating — even a small pre-negotiated overflow contract meaningfully changes what "no rider available" means operationally.
- **"What are the actual labor-compliance rules for our rider base (max continuous hours, mandatory breaks, right to reject)?"** These aren't optional constraints — they cap what the algorithm is legally/ethically allowed to do, and are needed before finalizing objective functions.

---

4. System Architecture: The Dispatch Pipeline



If the Matching Engine finds **zero idle riders** for a ready order, it doesn't fail — it hands the order to the **Escalation Ladder** (Section 6), which is a structured fallback sequence rather than a dead end.

5. The Matching Algorithm

5.1 Why “just assign the nearest idle rider” isn’t quite right

A naive greedy approach (assign each order to the closest/first idle rider, one at a time, as they arrive) is simple and often good enough at very small scale, but it’s **myopic** — it can leave a good rider-order pairing on the table because an earlier, slightly-suboptimal assignment used up the best rider first. At a single store with a handful of riders, this rarely matters much; it starts to matter more once you’re batching decisions across a full store queue.

5.2 The better approach: batch matching every few seconds

Rather than assigning the instant either an order or a rider becomes available, hold a short window (5-10 seconds) and solve the **assignment problem** across everyone available in that window simultaneously:

- **Nodes:** ready orders `R = {r1...rn}`, idle riders `D = {d1...dm}`
- **Cost matrix:** `C[i][j]` = predicted total delivery time if order *i* goes to rider *j*, plus penalty terms for capacity violations and fairness deviation
- **Objective:** minimize total system cost (or maximize a weighted efficiency+fairness score) subject to each rider taking at most *k* orders (capacity) and each order going to exactly one rider

This is solved via the **Hungarian algorithm** (classic bipartite matching) or, when riders can carry multiple orders, a **min-cost flow / small integer program**. At single-store scale (a handful of riders and orders), this solves in milliseconds — it’s genuinely cheap to do properly, so there’s little reason to settle for pure greedy once you’re past MVP.

5.3 Matching sophistication ladder

Approach	How it works	When it’s appropriate
FCFS / nearest-idle greedy	Assign each order to first available or closest rider	MVP, very low order volume
Batch bipartite matching (Hungarian/min-cost flow)	Solve optimally across a short time window	Standard for single-store dispatch at real scale
Reinforcement learning dispatch	Learns to assign considering downstream effects (e.g., not stranding a rider far from future demand)	City-wide, high-volume, centrally orchestrated systems (this is roughly what large ride-hailing/delivery platforms use, e.g. DiDi’s published dispatch research)
Auction/bid-based	Riders express preference/bid for orders	Platforms prioritizing rider choice and satisfaction over pure efficiency

5.4 The offer-accept-reject cascade

Even after the “optimal” match is computed, the rider can decline (or simply not respond). Standard pattern, same as ride-hailing apps:

1. Push the offer to the matched rider with a short response window (e.g., 15-30 seconds)
2. If declined or timed out, re-offer to the next-best rider from the same batch computation
3. If the local pool is exhausted, escalate outward (wider radius, then the shortage ladder in Section 6)

5.5 Batching multiple orders into one trip

When 2-3 ready orders have delivery addresses close to each other, assigning them all to one rider (rather than one each) improves fleet efficiency — as long as it doesn’t breach any individual order’s SLA. This requires:

- **Geographic clustering** of currently-pending orders (e.g., by geohash proximity)
- **Route sequencing** — a small traveling-salesman-style calculation to decide delivery order within the batch
- **SLA feasibility check** — reject a batching option if it would push any individual order past its promised time

6. When No Rider Is Available: The Escalation Ladder

This isn’t a single fallback — it’s a **graduated sequence**, where the system escalates only as far as necessary:

```
Step 1 → Queue order + show dynamic wait ETA
Step 2 → Predictive pipelining (pre-assign to soonest-returning rider)
Step 3 → Cross-store rider borrowing (neighboring dark store pool)
Step 4 → Surge/incentive trigger (pull nearby or off-duty riders)
Step 5 → Third-party fleet aggregator overflow
Step 6 → Demand throttling (temporarily slow new orders in that zone)
```

Step 1 — Queue with a dynamic wait estimate. Rather than freezing the customer's ETA, recompute it using the **predicted return time of the rider likely to free up soonest** — tracked from that rider's live GPS position and remaining route — rather than a generic average. This is more accurate than textbook queueing-theory averages (M/M/c-style models) at the level of a single store; aggregate queueing math is better used for macro capacity planning (how many riders to schedule this hour) than for the specific "who gets this order next" decision.

Step 2 — Predictive pipelining. Don't wait for a rider to be fully idle back at the store. If the system can predict which currently-busy rider will finish their drop-off soonest and is closest to being free, it can **pre-assign** the next order to them and route them straight back (or to the next pickup) the moment they complete their current delivery — effectively removing the "return to store" dead time from the critical path.

Step 3 — Cross-store rider borrowing. In dense urban areas, dark store delivery catchments often overlap. If dispatch is orchestrated centrally (not siloed per-store), an idle rider from a nearby store can be pulled to service this order — as long as doing so doesn't create a shortage at that neighboring store. This requires network-level visibility into rider pools across stores, not just within one.

Step 4 — Surge/incentive trigger. Automatically raise the per-order incentive in that zone to pull in off-duty gig riders or riders finishing nearby deliveries in adjacent zones. This should be **automatically triggered by real-time shortage signals**, not only manually set by ops — manual triggering introduces a meaningful response lag.

Step 5 — Third-party fleet aggregator overflow. If your own fleet is genuinely saturated, tap a pre-negotiated third-party on-demand delivery fleet as overflow capacity. This costs more per order but avoids a hard failure to deliver — worth having as a standing contract even if rarely used.

Step 6 — Demand throttling. As a last resort, temporarily slow down new order intake from that zone (raise minimum order value, hide the fast-delivery badge, show "high demand" messaging) rather than over-promising and breaching SLA broadly. This protects trust for customers who are already in the queue.

---

7. Factors You Didn't Mention — But That Matter

- **Fairness vs. pure efficiency in rider selection.** Always assigning the "best" (closest/fastest) rider can concentrate earnings on a small set of top riders and cause dissatisfaction/churn among the rest. Many real systems blend a fairness term (e.g., idle-time-weighted priority) into the cost matrix rather than optimizing purely for speed.
- **Mandatory rider breaks and labor compliance.** A rider marked "idle at store" might actually be on a compliance-mandated break — the system needs a distinct state for this, not just idle/busy, or it risks violating labor rules by auto-assigning through a break.
- **Rider self-reported availability being unreliable.** Without geofence verification, riders can mark themselves "at store" while elsewhere, leading to false-positive availability and wasted offer cycles.
- **Weather as a compounding factor.** Rain simultaneously reduces rider willingness to ride (supply shock) and increases order volume (demand shock) — the worst-case shortage scenario is exactly when both happen together, and the system should recognize this correlation rather than treating supply and demand as independent.
- **Trip chaining across stores.** If a rider's drop-off point happens to be near a *different* dark store than the one they started from, routing them to pick up their next order from that nearer store (rather than sending them back to their origin store) can meaningfully cut dead travel time — this requires network-level rather than store-siloed thinking.
- **Rider vehicle-order fit.** A bicycle rider shouldn't be matched to a large/heavy order that doesn't fit their bag capacity — this is a hard constraint on the matching, not just a preference.
- **Cold-chain/perishable priority in the queue.** When multiple orders are waiting for the next available rider, a frozen-goods order should generally jump the queue ahead of an equally-aged non-perishable order, even under pure FCFS logic.
- **New/low-rated rider ramp-up.** A newly onboarded rider may have less zone familiarity — worth factoring into the cost matrix (they may take longer on unfamiliar routes) rather than assuming uniform rider performance.
- **Rider proactive rejection patterns.** If a rider consistently rejects certain zones or order types, that's a signal worth feeding back into future matching (avoid wasting offer cycles on predictable declines) rather than re-offering blindly.

---

8. Constraints That Shape This System

Constraint Type	Specifics
Latency	Matching decisions should complete within a few seconds of an order becoming ready or a rider becoming idle — this is a real-time system, not a batch job
Vehicle/capacity	Max concurrent orders per rider is capped by bag/vehicle capacity — a hard constraint on any batching decision
Labor compliance	Max continuous working hours, mandatory breaks, and (in many gig-economy contexts) a rider's right to reject an offer without penalty
SLA/perishability	Cold-chain and promised-delivery-time constraints limit how long any order can sit in queue regardless of rider availability
Geofence accuracy	Rider "availability" is only trustworthy if location is verified, not self-declared
Fairness/earnings distribution	Pure efficiency-optimized dispatch can concentrate work on top riders, risking dissatisfaction and churn among the broader rider pool
Cost	Surge incentives and third-party aggregator overflow both cost more per order — needs a budget/threshold policy, not unlimited automatic spend
Network orchestration scope	Cross-store rider borrowing and trip-chaining are only possible if dispatch is centrally orchestrated rather than siloed per store
Rejection/timeout handling	Needs a bounded retry/escalation policy — offers can't cascade indefinitely without a defined ceiling before falling back further down the escalation ladder
Regulatory	Gig-worker regulations in many regions constrain forced assignment, minimum guaranteed pay, and insurance requirements — these cap what the algorithm can enforce

---

9. Base Assumptions

1. Riders are gig/flexible workers who can accept or reject offers (not fully captive employees who must accept whatever is assigned) — this shapes the need for an offer-cascade rather than a hard assignment.
2. Rider location and status tracking already exists at some basic level (even if not yet geofence-verified) — full build-from-zero location tracking would be a bigger lift than assumed here.
3. Dark stores in the same network have at least some geographically overlapping delivery catchments, making cross-store borrowing a realistic lever.
4. There's some existing (even if manual) incentive/surge mechanism for riders — fully automating its trigger is an enhancement, not a from-scratch build.
5. A single store's rider pool is small enough (single-digit to low double-digit concurrent riders) that exact optimal matching (Hungarian/min-cost flow) is computationally trivial in real time — the complexity is in the shortage-handling logic, not the matching math itself.

Document prepared as a system design framework for delivery-partner dispatch and supply-shortage handling in quick-commerce dark store operations.